The Mathematics of Secrets

Jayden Rios

November 2026

1 Introduction

The RSA cryptosystem uses number theory to create a means of encrypting and decrypting messages without requiring two parties to share a secret key, unlike symmetric cryptosystems. RSA is an impactful implementation of number theory and sheds light on some of the practical problems that can be solved from an abstract mathematical structure.

The foundations for RSA rely heavily on elementary number theory, including concepts like the Euclidean algorithm, Bézout's identity, modular inverses, Euler's totient function, and modular exponentiation. These ideas provide the mathematical framework for RSA key generation, encryption, and decryption.

An important aspect of this cryptosystem is that, even though messages can be encrypted and decrypted efficiently, factoring a sufficiently large and appropriately generated RSA modulus, which is the product of two primes, is computationally difficult using known classical algorithms. Integer factorization plays a crucial role in understanding the difficulty involved with recovering a private key.

This paper develops the mathematical foundations necessary for RSA, explores the computational complexity of key generation, encryption, and decryption, and examines integer factorization algorithms and their computational complexity. These two aspects are explored in order to exemplify the effectiveness of number-theoretic ideas elegantly combined to make impactful systems.

2 Mathematical Foundations

2.1 The Euclidean Algorithm and Bézout's Identity

As numbers become larger, it becomes extremely difficult to find their greatest common divisors. The remedy to this issue is the Euclidean Algorithm.

To prove this method, we begin with the assumptions that a is an integer and b is an integer greater than 0. The division algorithm asserts that there exists integers q and r such that

$$a = bq + r,$$

where

$$0 \leq r < b.$$

Suppose $d \mid a$ and $d \mid b$. Since

$$r = a - bq,$$

we know $d \mid bq$ and $d \mid a$. Therefore,

$$d \mid r.$$

Now let $d \mid b$ and $d \mid r$. Since

$$a = bq + r,$$

and $d \mid bq$ and $d \mid r$, it follows that $d \mid a$.

Because pairs (a, b) and (b, r) share exactly the same common divisors, they must have the same greatest common divisor. Hence, we have shown

$$\gcd(a, b) = \gcd(b, r).$$

If you let

$$d = \gcd(a, b),$$

where a and b are integers that are not both zero, there must be integers u and v such that

$$d = u \cdot a + b \cdot v.$$

The greatest common divisor of the two numbers can be expressed as a linear combination of a and b.

After finding the greatest common divisor of two integers using the Euclidean Algorithm, you may work backward through the Algorithm's equations, substituting each remainder into the previous equation, or so-called "reverse engineer" the equations. When you reach the first equation, you will find the greatest common divisor represented by a linear combination of the original arguments.

$$\gcd(252, 198)$$

$$252 = 1(198) + 54$$

$$198 = 3(54) + 36$$

$$54 = 1(36) + 18$$

$$36 = 2(18) + 0$$

$$\gcd(252, 198) = 18$$

$$18 = 54 - 36$$

$$18 = 54 - (198 - 3(54))$$

$$18 = 4(54) - 198$$

$$18 = 4(252 - 198) - 198$$

$$18 = 4(252) - 4(198) - 198$$

$$18 = 4(252) - 5(198)$$

$$\boxed{\gcd(252, 198) = 4(252) - 5(198)},$$

which show that 18 is written as a linear combination of 252 and 198 with the coefficients $(4, -5)$.

2.2 Modular Inverses

Before finding an inverse in the realm of a modulus, we must confirm that one exists. The proof of such goes as follows.

Assume that $m > 1$.

Proof. Suppose a has a multiplicative inverse modulo m; this means there exists an integer b such that

$$ab \equiv 1 \pmod{m}.$$

By definition of congruence, we have

$$m \mid (ab - 1).$$

Now let

$$d = \gcd(a, m),$$

so $d \mid a$ and $d \mid m$. $d \mid ab$ and $d \mid (ab - 1)$; it must divide their difference. Therefore,

$$d \mid ab - (ab - 1),$$

so

$$d \mid 1,$$

and that must mean that $d = 1$, so

$$1 = \gcd(a, m).$$

Conversely, suppose

$$\gcd(a, m) = 1.$$

By Bézout's identity, there exists integers p and q, s.t.

$$1 = pa + qm,$$

so, rearranging, gives

$$1 - pa = qm,$$

so

$$m \mid (1 - pa),$$

so

$$pa \equiv 1 \pmod{m},$$

thus p is the multiplicative inverse of $a \bmod m$.
Therefore, we have shown that

$$\boxed{\gcd(a, m) = 1 \iff a^{-1} \pmod{m} \text{ exists}}$$

□

These ideas become central in creating the RSA protocol, with their connection with the Euler phi function.

In order to choose our public exponent e, we must confirm

$$\gcd(e, \phi(n)) = 1.$$

This guarantees the existence of a multiplicative inverse modulo $\phi(n)$,

$$d \equiv e^{-1} \pmod{\phi(n)}.$$

More information on this will be provided later.

3 Efficient Modular Exponentiation

When we deal with numbers raised to a large power, direct computation becomes impractical. For these cases, we use efficient modular exponentiation; the method works as follows.

Suppose

$$x \equiv r \pmod{m}.$$

This means that they share the same residue class. Therefore, squaring both sides maintains their congruence, hence

$$x^2 \equiv r^2 \pmod{m}.$$

Furthermore, because reducing a number to its remainder also gives the same residue class, we can replace the intermediate products by their residues without changing the final residue. By applying this method, you are able to reduce each intermediate product of x and use powers of two, rather than painstakingly computing a huge power.

Any positive integer n can be expressed in binary form as

$$n = \sum_{i=0}^{k-1} b_i 2^i,$$

where k is binary bit length to represent n, and each

$$b_i \in \{0, 1\}.$$

We can use this to transform a number $a \in \mathbb{Z}$ with a power into binary digits. Replacing n to the summation shown before,

$$a^n = a^{\left(\sum_{i=0}^{k-1} b_i 2^i\right)},$$

which is equivalent to the product notation

$$\prod_{i=0}^{k-1} a^{b_i 2^i},$$

and because

$$b_i \in \{0, 1\},$$

so

$$a^{b_i 2^i} = \begin{cases} 1, & b_i = 0, \\ a^{2^i}, & b_i = 1. \end{cases},$$

which means the simplest form of the notation can be written as

$$a^n = \prod_{i:\, b_i=1} a^{2^i}.$$

Example:

Here we write 15 as a binary expression in order to make the exponentiation simpler to compute. We showed before that we may also reduce the intermediate products of a number by a modulus while maintaining its residue class. We will combine these two ideas in order to create a more computational efficient way of computing a larger power modulo m.

$$15 = 1111_2 = 2^3 + 2^2 + 2^1 + 2^0 = 8 + 4 + 2 + 1.$$

Therefore, by the laws of exponents, we have

$$a^{15} = a^8 a^4 a^2 a,$$

meaning

$$r_j \equiv a^j \pmod{m}, \qquad 0 \leq r_j < m,$$

$$a^{15} \equiv r_8 r_4 r_2 r_1 \pmod{m},$$

and when multiplying each intermediate product, we may reduce by the modulus. This makes computing the power much more manageable by repeated squaring and reducing each intermediate product modulo m.

This idea is what makes practical RSA computation plausible, because with this, you are able to multiply very large modular exponentiations without directly computing some possibly enormous powers. By understanding the squares of the base number, and reducing by the modulus, this is what makes computing very large exponents manageable. This lays the efficient computing foundation of RSA.

4 Number-Theoretic Foundations of RSA

4.1 Fermat's Little Theorem

Fermat's little theorem states that if we let p be prime, then any a satisfies

$$a^p \equiv a \pmod{p},$$

and any integer a not divisible by p satisfies

$$a^{p-1} \equiv 1 \pmod{p}.$$

The proof for this goes as follows.

Proof. Suppose that $p \nmid a$. We begin with the proposition that the integers $a, 2a, \ldots, a(p-1)$ are a complete set of residues $\pmod{p}$. Now suppose that two of them ia and ja are in the same residue class, i.e., $ja \equiv ia \pmod{p}$. This means that $p \mid (j-i)a$, but since we know $p \nmid a$, and p is prime, $p \mid (i-j)$. Furthermore, since $1 \leq i, j \leq p-1$, we have $|i-j| < p$. Because $p \mid (i-j)$, it is only possible that $i - j = 0$, so $i = j$.

Hence, the complete set of residue classes are merely a rearrangement of $1, 2, \ldots, (p-1) \pmod{p}$.

Thus, we can say that the product of the numbers in the first and second sequences are congruent, i.e., $a^{p-1}(p-1)! \equiv (p-1)! \pmod{p}$, which is equivalent to $(p-1)!(a^{p-1}-1) \equiv 0 \pmod{p}$, so $p \mid (p-1)!(a^{p-1}-1)$, and since we know $p \nmid (p-1)!$, then $p \mid (a^{p-1}-1)$, thus $a^{p-1} \equiv 1 \pmod{p}$. Multiplying both sides by a, we get $a^p \equiv a \pmod{p}$.

Finally, if $p \mid a$, then $a \equiv 0 \pmod{p}$. Thus, $a^p \equiv 0 \pmod{p}$ also, so again $a^p \equiv a \pmod{p}$, concluding the proof. □

An extension of Fermat's little theorem goes as follows: if $p \nmid a$, and if $n \equiv m \pmod{p-1}$, then $a^n \equiv a^m \pmod{p}$.

Suppose first that $m \leq n$, so $c \geq 0$.

Proof. Suppose $p \nmid a$ and $n \equiv m \pmod{p-1}$. This means that $(p-1) \mid (n-m)$. It follows that there exists some integer c such that $c(p-1)+m = n$. Now, if we have a^n, we can see that $a^n = a^{c(p-1)+m} = a^{c(p-1)}a^m$, and by Fermat's little theorem, if $p \nmid a$, $a^{p-1} \equiv 1 \pmod{p}$. Therefore, $a^{c(p-1)} \equiv (a^{p-1})^c \equiv 1^c \pmod{p}$, hence $a^n \equiv a^m \pmod{p}$. □

If $m > n$, the same argument applies by interchanging m and n.

To illustrate the impact of Fermat's theorem, we investigate the last base 5 digit of 3^{123}. This is equivalent to $3^{123} \pmod{5}$. Because we know

$$123 \equiv 3 \pmod{4},$$

once we reduce the exponent modulo 4, we may observe that

$$3^{123} \equiv 3^3 \equiv 27 \equiv 2 \pmod{5}.$$

4.2 Euler's Totient Function and the Chinese Remainder Theorem

The Euler phi-function $\phi(n)$ is defined as the number of nonnegative integers $b < n$ that are relatively prime to n:

$$\phi(n) = |\{0 \leq b < n : \gcd(b, n) = 1\}|.$$

In RSA, the modulus we work with is $n = pq$, where p and q are distinct primes. Because $\phi(n)$ is multiplicative for components that are relatively prime,

$$\phi(n) = \phi(pq) = \phi(p)\phi(q) = (p-1)(q-1).$$

The Chinese Remainder Theorem is what allows for the multiplicative property. Because p and q are relatively prime, this means that for every pair of residue classes one from modulo p, the other from modulo q, there is a unique residue class modulo pq. A residue a is relatively prime to pq exactly when it is relatively prime to both p and q. Therefore, the number of such residues present modulo pq is the product $\phi(p)\phi(q)$.

To establish the mathematical correctness of RSA, we must first state Euler's theorem, which is a generalization of Fermat's little theorem. The theorem allows for simplification of powers in modular arithmetic, which is an essential component for why RSA decryption uncovers the original message. The theorem states that if $a \in \mathbb{Z}, n > 1$, and $\gcd(a, n) = 1$, then $a^{\phi(n)} \equiv 1 \pmod{n}$.

5 RSA Key Generation and Correctness

5.1 Key Generation

Now for the construction of RSA, we begin with the definition of its characteristic variables. The user must first select two large distinct prime numbers; let these be p and q. Now let $n = p \cdot q$, and from what we previously discovered we can find the totient of this product as follows: $\phi(n) = (p-1) \cdot (q-1)$.

We now choose a number e which is relatively prime to the totient $\phi(n)$. Furthermore, we assume that $1 < e < \phi(n)$.

Because we know $\gcd(e, \phi(n)) = 1$, then there exists some integers d and k such that $ed + k\phi(n) = 1$, and if we reduce modulo $\phi(n)$, we get $ed \equiv 1 \pmod{\phi(n)}$. Thus, we obtain d that is the multiplicative inverse of e modulo $\phi(n)$, which is the private component used in RSA decryption. It must be kept in mind that Bezout's identity guarantees an integer d, but this integer may be negative, and we need to choose a positive representative $d \pmod{\phi(n)}$. We now have our public key, which is (n, e), and our private key (n, d), with d, p and q also remaining secret.

5.2 Correctness of RSA

Now for RSA, if we assume that $\gcd(M, n) = 1$, Euler's Theorem guarantees $M^{\phi(n)} \equiv 1 \pmod{n}$. We also just showed that $ed \equiv 1 \pmod{\phi(n)}$, so for some integer $k \geq 0$, $ed = 1 + k\phi(n)$. Now we can see that if we encrypt then decrypt a posed message M, we get

$$\begin{aligned} M^{ed} &= M^{1+k\phi(n)} \\ &= M(M^{\phi(n)})^k \\ &\equiv M \pmod{n}. \end{aligned}$$

In RSA, e and d are chosen such that $ed \equiv 1 \pmod{\phi(n)}$, and since we have shown that $\phi(n) = (p-1)(q-1)$, we can see that $p-1$ and $q-1$ both divide $\phi(n)$. Therefore, $ed \equiv 1 \pmod{p-1}$ and $ed \equiv 1 \pmod{q-1}$.

The previous argument establishes RSA correctness only when $\gcd(M, n) = 1$; the following proof generalizes this to all integer messages M, including those that are not relatively prime to n.

Suppose $ed \equiv 1 \pmod{p-1}$, $ed > 0$, and p is prime. This means that $p - 1 \mid ed - 1$, so there exists some integer z such that $z(p-1) = ed - 1$, so $z(p-1) + 1 = ed$. So now

$$M^{ed} \equiv M^{(p-1)z+1} \equiv (M^{p-1})^z M \pmod{p}.$$

Now we split into two cases.

Case 1: If $p \nmid M$, then we know that by Fermat's Little Theorem $M^{p-1} \equiv 1 \pmod{p}$, so $(M^{p-1})^z M \equiv (1)^z M \pmod{p}$, and since 1 raised to any power is itself, the congruence becomes $M \pmod{p}$, hence $M^{ed} \equiv M \pmod{p}$.

Case 2: If $p \mid M$, then we know that $M \equiv 0 \pmod{p}$, so $M^{ed} \equiv 0 \pmod{p}$, therefore $M^{ed} \equiv M \pmod{p}$.

The same argument is applied to modulo q. Because $M^{ed} \equiv M \pmod{p}$ and $M^{ed} \equiv M \pmod{q}$, the Chinese Remainder Theorem implies $M^{ed} \equiv M \pmod{pq}$, and since $n = pq$, $M^{ed} \equiv M \pmod{n}$, which establishes the correctness of RSA decryption for all integer messages M.

5.3 Encryption and Decryption Example

To more clearly illustrate the encryption and decryption process, we will consider small prime numbers for simplicity. Let $p = 17$ and $q = 11$.

Next, we form the modulus n, which is

$$n = 17 \cdot 11 = 187,$$

and Euler's totient

$$\phi(n) = (11-1)(17-1) = (10)(16) = 160.$$

Now we choose a number e, which is relatively prime to the totient, let this be $e = 7$, since

$$\gcd(7, 160) = 1.$$

Now for the private exponent, we find the multiplicative inverse of e modulo $\phi(n)$,

$$ed \equiv 7(23) \equiv 1 \pmod{160},$$

therefore $d = 23$.

We have now constructed the public and private keys, respectively:

$$E = (e, n) = (7, 187),$$

and

$$D = (d, n) = (23, 187).$$

We may now use the keys to encrypt then recover the message "DOG". Use a numbered alphabet, beginning with 0, to get the equal number values. Each number is converted to the numerical equivalents using $A = 0, B = 1, \ldots, Z = 25$. For this example, each letter will be encrypted independently, although RSA can also encrypt numerical blocks that represent multiple letters.

$$DOG = (3, 14, 6).$$

Our encryption elements are $E = (7, 187)$, so

$$C \equiv 3^7 \equiv 130 \pmod{187},$$

making the encryption elements $(130, 108, 184)$. Now, to decrypt, we use $D = (d, n)$, so

$$M' \equiv C^d \equiv 130^{23} \equiv 3 \pmod{187},$$

and translating this back into its letter equivalent, we observe that the decryption recovers $(3, 14, 6)$, which correspond to the letters DOG. Therefore, we have recovered the enciphered text.

6 Computational Complexity

6.1 Bit Length

A positive integer has exactly k binary digits when

$$2^{k-1} \leq n < 2^k.$$

To find the bit representation of an integer, we begin by applying $\log_2$ to the inequality.

$$k - 1 \leq \log_2 n < k.$$

We can apply the floor function to determine the number of bits in the binary representation of an integer n. Here we can see that $\lfloor \log_2 n \rfloor = k - 1$, rearranging gives $k = \lfloor \log_2 n \rfloor + 1$. This shows that the bit length grows logarithmically with the magnitude of an integer. Therefore, if we were to double the magnitude of an integer it would increase the bit length of the integer by exactly 1 bit, because we are dealing with the $\log_2$.

We measure computational complexity using elementary bit operations. Larger integers require more bits to be represented. Consequently, arithmetic operations on larger numbers generally requires more elementary bit operations.

6.2 Binary Arithmetic Operations

To find the bit complexity of addition, we consider how addition is executed. When two integers are converted to binary, consider their numbers arranged by digit. If either of the first digits on the top or bottom contain a one and the other contains a 0, then the 1 is written for the first digit. Now, if both digits are 1, then the 1 is carried to the next digit and the result of that column is 0, assuming there is no carry. If there is, in fact, an incoming carry, the resulting digit is instead 1, and the other carry of 1 is still passed onto the next digit.

Because processing each column requires $O(1)$ computations, there will be at most k input columns to process, with an additional bit to carry; therefore, the total number of bit operations as a function grows linearly with k, so the number of bit operations is $O(k)$.

An example of this is shown as follows. Consider the bit representations of 20 and 15, which are 10100 and 1111 respectively.

$$\begin{array}{r} 10100 \\ +\,01111 \\ \hline 100011 \end{array}$$

In order to express the bit complexity of multiplication, we examine how the multiplication algorithm works. Consider multiplying the bit representation of two numbers; if the bottom contains a 1, the partial product is a copy of the top number. If the number on that column is 0, then the corresponding row is all 0's.

This first process takes $O(k)$ bit operations, but now each partial product has to be added in order to create a result, and we know that multiplying two k bit integers produces at most k partial product rows; each row is then incorporated into the running sum using $O(k)$ elementary bit operations. Hence the fact that multiplying the number of rows by the cost per row gives the total complexity of

$$O(k) \cdot O(k) = O(k^2).$$

An example of this is shown as follows.

$$\begin{array}{r} 10100 \\ \times\ 1111 \\ \hline 10100 \\ 101000\ . \\ 1010000 \\ 10100000 \\ \hline 100101100 \end{array}$$

6.3 Computational Cost of RSA

The computational complexity of arithmetic is especially important for RSA because its encryption and decryption elements involve repeated modular multiplication.

RSA encryption computes $C = M^e \pmod{n}$, while decryption computes $M = C^d \pmod{n}$. Furthermore, to reference what we previously found, we know that an integer n has a bit representation of $k = \lfloor \log_2(n) \rfloor + 1$. This connects to the earlier result found, which is that the classical model of multiplication requires $O(k^2)$ bit operations. The cost of each arithmetic operation lays the foundation for understanding the computational cost of RSA encryption and decryption.

Let k be the number of bits required to represent the modulus n, and t be the number of bits required to represent e or d. Repeated squaring works by observing binary digits of an exponent. Each binary digit requires one squaring and based on its digit, also one additional multiplication. This means that an exponent containing t bits requires at most $2t$ modular multiplications, meaning it has a complexity of $O(t)$ modular multiplications.

We previously established that multiplying two k bit integers requires $O(k^2)$ bit operations. Because these integer representations of intermediate results are each reduced by a modulus n, their values remain bounded by a modulus; this allows each modular multiplication to be completed in $O(k^2)$ bit operations, under a classical arithmetic model.

Combining these two ideas together, we form the representation of the computational complexity of modular exponentiation

$$O(t) \cdot O(k^2) = O(tk^2).$$

In RSA, the exponents can be chosen to have at most k bits, so $t = O(k)$; substituting into the expression above gives

$$O(tk^2) = O(k \cdot k^2) = \boxed{O(k^3)}.$$

Because complexity grows polynomially to the bit length of the modulus, RSA encryption and decryption can be performed efficiently.

7 Integer Factorization and RSA Security

The security of RSA depends on the computational complexity of factoring a large number, which is given as $n = pq$, where p and q are distinct primes. An attacker is able to uncover the private element if they can calculate $\phi(n) = (p-1)(q-1)$ and subsequently recover $d \equiv e^{-1} \pmod{\phi(n)}$.

7.1 Recovering Factors from Euler's Totient

For RSA, it is important to understand that an attacker knowing $\phi(n)$ gives them enough information to factor n, so they can recover p and q as follows.

Since $n = pq$ and $\phi(n) = (p-1)(q-1)$, we have $\phi(n) = pq - p - q + 1$, so $\phi(n) = n - p - q + 1$. Rearranging gives $p + q = n + 1 - \phi(n)$, meaning we know the sum of the two primes, and the product is given $n = pq$. Once you know both the product and the sum of two primes, you can recover those two numbers by identifying the roots of a quadratic equation, found by their sum and product.

Let x be an arbitrary value, and because we know the sum and product of the elements p and q, we can construct the equation

$$x^2 - (n + 1 - \phi(n))x + n = 0.$$

Plugging into the quadratic formula gives

$$x = \frac{(n + 1 - \phi(n)) \pm \sqrt{(n + 1 - \phi(n))^2 - 4n}}{2}.$$

Since we assume $p > q$, we know that the positive gives p, while the negative gives q. Thus, knowing n and $\phi(n)$ allows us to find the factors p and q, which are the prime factors of n. Although n is public, $\phi(n)$ is not publicly known and cannot be obtained from n without additional computation.

7.2 Trial Division

One of the simplest ways an attacker can attempt to factor n is through pure trial division. The division of the trial iteratively tests which successive integers divide n without remainder. We only need to test the numbers up to $\sqrt{n}$, because we assume that $n = pq$. For contradiction, assume that $p > \sqrt{n}$ and $q > \sqrt{n}$, multiplying gives $pq > \sqrt{n}\sqrt{n} = n$, but since $pq = n$, we know that this is a contradiction. Therefore, at least one of the divisors must be less than or equal to $\sqrt{n}$. If we test each of these successive integers for trial division, we see that the computational complexity is $O(\sqrt{n})$ in the worst case. As we explored before the input size is measured by the length of binary digits, which is $k = \lfloor \log_2 n \rfloor + 1$. Since $n \approx 2^k$, we can see that $\sqrt{n} \approx 2^{k/2}$, meaning that the worst case complexity in terms of bit length is $O(2^{k/2})$, which represents the number of candidate divisors tested. This shows that when the moduli become increasingly large, this method becomes impractical, because of increasingly large amount of time it takes to compute a factor.

7.3 Pollard's Rho Algorithm

One of the other ways in which an attacker can uncover private keys was previously shown, if they knew $\phi(n)$, which showed the value of knowing a public component. Another more realistic way of uncovering the private key is by Pollard's rho algorithm.

First, we choose a polynomial, which is typically

$$f(x) = x^2 + 1 \pmod{n},$$

and begin with x_0, generate $x_{j+1} = f(x_j) \pmod{n}$. Compare the iterations x_j, y_j, which are calculated by

$$x_{j+1} = f(x_j) \pmod{n}$$

and

$$y_{j+1} = f(f(y_j)) \pmod{n},$$

which both begin at the same value, $x_0 = y_0$; we then calculate $g_j = \gcd(|x_j - y_j|, n)$.

Now we have 3 conditions: if $1 < g_j < n$, then we have found a factor of n; if $g_j = 1$, no factor has been found from that comparison; and if $g_j = n$, then the comparison failed to produce a useful factor.

This works because two iterations may satisfy $x_j \equiv y_j \pmod{p}$ for an unknown factor $p|n$, even though they differ modulo n; therefore $p|x_j - y_j$. Now also suppose that these values aren't congruent modulo q,

i.e., $x_j \not\equiv y_j \pmod{q}$. Consequently, because $p|(x_j - y_j)$, while $q \nmid (x_j - y_j)$. Since $n = pq$, it follows that $\gcd(|x_j - y_j|, n) = p$.

An example of this goes as follows:

Suppose we want to factor $n = 91$. We begin by choosing a function $f(x) = x^2 + 1 \pmod{91}$. We now generate the sequence using $x_{j+1} = f(x_j) \pmod{91}$. The generated sequence is

$$x_0 = y_0 = 1, x_1 = f(1) = 2, y_1 = f(f(1)) = f(2) = 5$$

We can now compare the values using the GCD, and we find that $\gcd(|2 - 5|, 91) = 1$, which means this iteration is not a factor, so we continue.

$$x_2 = f(2) = 5, y_2 = f(f(5)) = f(26) = 40$$

For this pair, we have $\gcd(|5 - 40|, 91) = 7$, which is a nontrivial factor; therefore $91 = 7 \cdot 13$. Since we have found one factor $p = 7$, we can find the other $q = 91/7 = 13$; thus $91 = 7 \cdot 13$.

Once we choose x_0 and the function $f(x)$, every subsequent value is completely determined, so we can see that there is no randomness involved. Although this is true, factoring behavior can be complicated enough that we may analyze it as if its values were approximately randomly distributed modulo p. Under this randomness behavior, the sequence modulo p is expected to experience collision after approximately $O(\sqrt{p})$ iterations.

For RSA, assuming that $p \approx q \approx \sqrt{n}$, shows that the count is $O(\sqrt{p}) \approx O(\sqrt{\sqrt{n}}) = O(n^{1/4})$, where $O(\sqrt{p})$ is an expected iteration. As we stated before, an integer n converted to bits is $n \approx 2^k$, allowing for us to obtain $O(n^{1/4}) = O(2^{k/4})$. We saw before that trial division requires up to $O(2^{k/2})$ divisibility checks. Thus, we can see that Pollard's rho has a much smaller growth estimate.

8 Conclusion

RSA largely relies on the mathematical structures of the Euclidean algorithm, Bézout's identity, modular inverses, and Euler's totient function. These concepts work together to make key generation, encryption, and decryption possible. These foundations are what allow for the RSA correctness proofs, which show how RSA decryption recovers original messages.

The processes and underlying structure of RSA demonstrate how mathematical concepts can be combined into a sophisticated and practical system. Its security is dependent on more than mathematical correctness, which is why computational difficulty is also essential. The computational complexity of integer factorization illustrates the challenges faced when attempting to recover private keys. Understanding RSA is understanding the relationship between pure mathematics, algorithms, and real-world applications.

References

[1] N. Koblitz, *A Course in Number Theory and Cryptography*, 2nd ed., Springer, 1994.

[2] R. L. Rivest, A. Shamir, and L. Adleman, A Method for Obtaining Digital Signatures and Public-Key Cryptosystems, Communications of the ACM, 21(2), 120–126, 1978.

[3] J. Hoffstein, J. Pipher, and J. H. Silverman, *An Introduction to Mathematical Cryptography*, 2nd ed., Springer, 2014.